

xplainfi: Feature Importance and Statistical Inference for Machine Learning in R

by Lukas Burk, Fiona Katharina Meinzer, Giuseppe Casalicchio, Marvin N. Wright, and Bernd Bischl

Abstract We introduce `xplainfi`, an R package built on top of the `mlr3` ecosystem for global, loss-based feature importance methods for machine learning models. Various feature importance methods exist in R, but significant gaps remain, particularly regarding conditional importance methods and associated statistical inference procedures. The package implements permutation feature importance, conditional feature importance, relative feature importance, leave-one-covariate-out, and generalizations thereof, and both marginal and conditional Shapley additive global importance methods. It provides a modular conditional sampling architecture based on Gaussian distributions, adversarial random forests, conditional inference trees, and knockoff-based samplers, which enable conditional importance analysis for continuous and mixed data. Statistical inference is available through multiple approaches, including variance-corrected confidence intervals and the conditional predictive impact framework. We demonstrate that `xplainfi` produces importance scores consistent with existing implementations across multiple simulation settings and learner types, while offering competitive runtime performance. The package is available on CRAN and provides researchers and practitioners with a comprehensive toolkit for feature importance analysis and model interpretation in R.

1 Introduction

In machine learning (ML), understanding feature-target relationships is increasingly valued alongside predictive accuracy. Complex models can capture nonlinear patterns but are typically considered “black box” models due to their opaque internal mechanisms, which has motivated the development of interpretable machine learning (IML) methods (Molnar, 2020; Murdoch et al., 2019). Among these, feature importance (FI) methods aim to quantify the relevance of input features for a given prediction target, such as a fixed model, the learning algorithm that produced it, or the underlying data-generating process, thereby providing insight into what drives predictive behavior (Murdoch et al., 2019; Fisher et al., 2019). The applications range from increasing our understanding of a given data-generating process (DGP) to practical issues such as feature selection (Guyon and Elisseeff, 2003; Guidotti et al., 2018).

We present `xplainfi`, an R package that provides a unified interface for computing and comparing global, loss-based FI methods, which measure the change in predictive performance when features are removed, perturbed, or marginalized (Ewald et al., 2024). Alternative approaches out of scope include local FI methods (i.e., FI values for individual predictions such as Shapley values (see Shapley, 1953; Rozenberczki et al., 2022)) and variance-based sensitivity measures, such as Sobol indices (Sobol', 2001). When we refer to FI methods from here on, we mean global, loss-based FI methods unless otherwise stated. Various FI methods are implemented across packages in both the R and Python ecosystems, but in the R ecosystem, some methods are completely absent. `xplainfi` aims to fill this gap by providing a larger collection of FI methods than previously available, along with p-values and confidence intervals for FI uncertainty quantification, which are often needed in applications. The package is built on top of the `mlr3` ecosystem (Lang et al., 2019), which allows development to focus on the FI methods themselves without re-implementing common building blocks such as abstractions for learning algorithms, resampling, tuning, and pipelines (Binder et al., 2021).

Because many real-world datasets exhibit dependent features, marginal, perturbation-based FI methods suffer from two distinct issues. First, breaking the dependence structure forces the model to extrapolate to implausible feature combinations, potentially yielding unreliable predictions and hence FI values (Hooker et al., 2021). Second, importance is often distributed across dependent predictors (if they are reflected in the model), which is not necessarily a problem, but users need to be aware of it at the very least (Nicodemus et al., 2010; Debeer and Strobl, 2020). Conditional importance methods address this by assessing performance changes under interventions that preserve feature dependencies (Watson and Wright, 2021), yet remain underrepresented in available implementations. One challenge with these methods is the need for suitable conditional sampling methods, which is why `xplainfi` provides multiple methods for continuous and mixed data. Both conditional importance methods and the ability to handle mixed data are motivated by common issues arising in practical statistical work and data analysis, and they are also areas of active research (see also Blesch et al., 2024; Redelmeier et al., 2020). To that end, we offer `xplainfi` as a comprehensive, extensible tool to support these efforts.

Contributions

xplainfi provides a unified framework for FI methods built on the **mlr3** ecosystem. Key contributions include: 1) Implementation of many standard FI methods, including permutation feature importance (PFI) (Breiman, 2001), conditional feature importance (CFI) (Strobl et al., 2008; Debeer and Strobl, 2020), leave-one-covariate-out (LOCO) (Lei et al., 2018), relative feature importance (RFI) (König et al., 2021), and both marginal and conditional SAGE (Shapley Additive Global importance) (Covert et al., 2020); 2) Seamless integration with **mlr3**'s learners, tasks, measures, and resampling strategies; 3) A modular conditional sampling interface supporting Gaussian, adversarial random forests (ARF) (Watson et al., 2023), conditional inference trees (Hothorn et al., 2006), and knockoff-based samplers (Candès et al., 2018); 4) The option to target model, learner, or DGP importance by leveraging **mlr3**'s support for resampling, ensembling, and tuning; 5) Uncertainty quantification via variance-corrected confidence intervals (Nadeau and Bengio, 2003), observation-level LOCO inference (Lei et al., 2018), and the CPI testing framework (Watson and Wright, 2021).

The paper is structured as follows: First, we give a brief overview of the implemented global, loss-based FI methods and ways to quantify their uncertainty in Section 2. In Section 3, we present other R and Python packages that implement FI methods. Section 4 gives an introduction to **xplainfi** and showcases core functionality. Example scenarios, along with comparisons to existing implementations based on FI values and runtime, are provided in Section 5. We conclude with a discussion and outlook on **xplainfi** in Section 6.

Reproducibility and availability: **xplainfi** is on CRAN and maintained on GitHub. The GitHub repository contains all the code required to reproduce the results. The code and results are also included in the online supplement.

2 Implemented feature importance methods

We briefly review the FI methods implemented in **xplainfi**, drawing on the comprehensive overview in Ewald et al. (2024), to which we refer for a broader discussion and additional references. We also summarize the statistical inference procedures supported by the package. Rather than providing a full theoretical treatment, we focus on the estimands and quantities that are required for computation and implementation.

Notation: We assume the supervised learning setting with n observations of p features $X = (X_1, \dots, X_p)$ and a target Y . A model $\hat{f}$ is trained to predict Y from X , and its performance is evaluated using a loss function $L(Y, \hat{f}(X))$. The *feature of interest* (FOI) is denoted X_j , with X_{-j} representing all remaining features. More generally, X_S denotes a subset of features indexed by $S \subseteq \{1, \dots, p\}$. Feature importance for feature j is denoted FI_j , with specific methods indicated by subscripts (e.g., PFI_j , $LOCO_j$).

One central distinction between FI methods is whether they target *marginal* (unconditional) association between a feature and the target, $X_j \not\perp\!\!\!\perp Y$, or *conditional* association given a set of features G , $X_j \not\perp\!\!\!\perp Y \mid X_G$ (Strobl et al., 2008; Watson and Wright, 2021). Many well-established model-independent methods target marginal association (e.g., correlation- or information-theoretic measures) (Li et al., 2018; Bommert et al., 2020).

In the paper and **xplainfi**, we primarily focus on conditional association of X_j with Y , often with the important special case of conditioning on all remaining features X_{-j} . This captures the *incremental* predictive value of a feature when the others are already available, and it entails a substantially harder estimation problem than marginal association, typically requiring predictive models and procedures that respect the joint structure of the features. Conditioning on arbitrary feature subsets is also supported by **xplainfi**, e.g., via relative feature importance (RFI) (see Section 2.2). Importantly, the loss-based FI methods considered below quantify *predictive* importance (or reliance) under a particular intervention scheme (e.g., permutation, conditional resampling, refitting, or marginalization) and are therefore not equivalent to generic measures of statistical dependence. For certain population-level (oracle) importance parameters—including refitting-based measures in the framework of Williamson et al. (2023) under suitable losses—a zero importance parameter can be equivalent to conditional independence, but FI does not imply a causal effect of X_j on Y , and estimated FI values can depend on the chosen intervention distribution as well as the presence of correlated or redundant predictors.

2.1 Estimands and statistical inference

There are three possible targets of an FI analysis, corresponding to distinct estimands: a fitted *model*, a given *learner*, or the true prediction function regarding the DGP, also called population-level inference (see also Chiaburu et al., 2024; Molnar et al., 2023). *Model importance* analyzes a single, fixed model

and quantifies how it uses features for prediction. *Learner importance* targets the learning algorithm which produces the model when training data is sampled from the DGP of a given sample size. Unlike model importance, it captures how much models of this learner class rely on each feature on average, and requires refitting via resampling across multiple model instantiations on different test data. *DGP importance* aims to explain the true feature-target relationship by analyzing the population-level prediction function (or Bayes-optimal predictor), e.g., $f_0(x) = \mathbb{E}[Y | X = x]$ under ℓ_2 loss. As we usually never have direct access to this Bayes-optimal predictor in practical applications, this function must then be approximated by a strong learner in an AutoML-like fashion, e.g., by optimizing over multiple model classes and hyperparameters (Thornton et al., 2013) or by constructing heterogeneous stacking ensembles (Erickson et al., 2020; van der Laan et al., 2007). In practice this is very similar to *learner importance*, while model building is usually more expensive.

Statistical inference can be performed at each of the three estimand levels. At the model level, hypothesis tests can be based on observation-wise losses on a single test set. At the learner level, variance-corrected tests can be applied to paired loss differences across resampling iterations. Since FI estimation inherently involves random components (e.g., from data resampling, permutation, or stochastic learners), repeated evaluation is generally needed to quantify this uncertainty, just as cross-validation is needed for reliable performance estimation. `xplainfi` provides dedicated inference methods at the model and learner levels, which therefore also include population-level inference.

When evaluating multiple FOIs simultaneously, any of these inference procedures introduces a multiple-testing problem. Depending on the use case, it is necessary to control either the family-wise error rate (FWER) via methods such as the Bonferroni-Holm correction, or to control the false-discovery rate (FDR) with methods such as the Benjamini-Hochberg or Benjamini-Yekutieli procedures, where the latter is valid under arbitrary dependence structures (Holm, 1979; Benjamini and Hochberg, 1995; Benjamini and Yekutieli, 2001). In an exploratory setting, it is often sufficient to control the FDR, which controls the expected proportion of false positives among all rejections (e.g., 1 out of 20 features deemed important), whereas controlling the FWER is more suitable for confirmatory settings in which avoiding false positives takes priority.

Regardless of the estimand, loss-based FI methods are usually constructed by “removing” the information about the FOI(s) and calculating the difference in expected loss (with vs. without this information). Following Ewald et al. (2024), we group the FI methods by the strategy used to remove information: feature perturbations, model refitting, or Shapley values.

2.2 Methods based on feature perturbations

The most well-known method in this category is *permutation feature importance* (PFI), originally introduced by Breiman (2001) for random forests. PFI targets the effect of breaking the link between X_j and the rest of the joint distribution at evaluation time, holding $\hat{f}$ fixed. The model is fit once but evaluated twice: once with the original features and once with the FOI X_j replaced by a perturbed version $\tilde{X}_j$. In PFI, $\tilde{X}_j$ is derived by simply shuffling X_j in-place, which keeps the marginal distribution of X_j intact. Importance is then measured as the difference in performance between the original model prediction and the model prediction using the uninformative replacement for the FOI. Formally,

$$\text{PFI}_j = \mathbb{E} \left[L \left(Y, \hat{f}(\tilde{X}_j, X_{-j}) \right) \right] - \mathbb{E} \left[L \left(Y, \hat{f}(X) \right) \right]. \quad (1)$$

While PFI is computationally cheap, requiring only model predictions and simple shuffling of feature vectors, it breaks dependencies not only between the FOI and the target, but also between the FOI and all other features. This can yield implausible feature combinations and misleading attributions under feature dependence (see Hooker et al., 2021). *Conditional permutation feature importance* (CFI), introduced by Strobl et al. (2008), addresses this by perturbing the FOI conditional on a set of other features: X_j is replaced by $\tilde{X}_j$ sampled from the conditional distribution of $X_j | X_{-j}$, thereby preserving (parts of) the dependence structure among the predictors. König et al. (2021) generalize PFI and CFI to *relative feature importance* (RFI) by specifying a conditioning set G for which $\tilde{X}_j$ retains conditional dependencies, i.e., $\tilde{X}_j$ is sampled from the conditional distribution of $X_j | X_G$; this yields CFI for $G = -j$ and PFI for $G = \emptyset$.

Since the permutation (or sampling) step introduces randomness, multiple repetitions should be used to stabilize the estimate of the importance score. In `xplainfi`, this is controlled via the `n_repeats` parameter, with the final importance score averaged across repetitions. In terms of inference, `xplainfi` allows applying the corrected t-test approach proposed by Nadeau and Bengio (2003) to paired loss differences across resampling iterations, which can be used to quantify uncertainty of (learner-level) FI estimates based on repeated data splits. Molnar et al. (2023) evaluate and recommend this approach for PFI in combination with subsampling or bootstrapping with 10–15 iterations. Schulz-Kämpel et al. (2025) recommend for this test a train-test ratio of 0.9 and 25 subsampling iterations. `xplainfi` lets the

user apply the inference method to all FI methods, while warning the user if fewer than 10 bootstrap or subsampling iterations are used.

Beyond conditional permutation-based approaches, the conditional predictive impact (CPI) framework proposed by [Watson and Wright \(2021\)](#) provides a dedicated hypothesis test for conditional feature importance. [xplainfi](#) offers both the original version based on the knockoff framework ([Candès et al., 2018](#)) and an ARF-based version for mixed data ([Blesch et al., 2025](#)).

The conditional sampling required by CFI, RFI, and conditional SAGE (described below) can be performed using different approaches, each with distinct trade-offs. The Gaussian approach assumes multivariate normality, making it fast and straightforward but limited to continuous features. Conditional inference trees provide a nonparametric alternative that handles mixed feature types, as proposed by [Redelmeier et al. \(2020\)](#) for conditional distribution estimation (e.g., for conditional Shapley-style estimands) and implemented in [shapr](#). Adversarial random forests offer flexible density estimation and conditional sampling for mixed data at a higher computational cost ([Watson et al., 2023](#)). Finally, knockoff sampling provides a specialized approach that enables valid inference through the CPI framework, but implementations for mixed data are less common ([Candès et al., 2018](#); [Blesch et al., 2024](#)). Gaussian, conditional inference tree, and ARF samplers can be applied to all conditional importance methods in [xplainfi](#), while knockoff sampling is only compatible with CFI.

2.3 Methods based on model refitting

A more intuitive approach to remove an FOI's information is to refit a model without the FOI (using the same learner) and then measure the performance difference between the reduced and the original (full) model. The best-known method following this approach is the *leave-one-covariate-out* (LOCO) ([Lei et al., 2018](#)), which analyzes a single FOI X_j . If one is interested in the importance of a set of features at once, this is known as *leave-one-group-out* (LOGO) ([Au et al., 2022](#)). [Williamson et al. \(2023\)](#) proposed a general framework for refitting-based methods, including a dedicated statistical inference method. In [xplainfi](#), we use the terms LOCO and WVIM (*Williamson's variable importance measure*; [Ewald et al. \(2024\)](#) proposed the acronym).

This approach is conceptually simple: To evaluate an FOI's importance, the model in question is fit twice, once with and once without the FOI. Again, the resulting performance difference gives a straightforward indication of the predictive value "lost" by leaving out feature X_j :

$$\text{LOCO}_j = \mathbb{E} \left[L \left(Y, \hat{f}_{-j}(X_{-j}) \right) \right] - \mathbb{E} \left[L \left(Y, \hat{f}(X) \right) \right], \quad (2)$$

where $\hat{f}_{-j}$ denotes the model fitted without feature X_j . Model refitting, of course, increases the computational cost in comparison to FI methods based on perturbations, where only one model needs to be fit as the basis for all subsequent calculations. This can be problematic when using LOCO (or a generalization thereof) on datasets with many features, especially when combined with expensive learners.

The inverse operation, *leave-one-covariate-in* (LOCI), trains models with only a single feature and compares performance against a featureless baseline to measure the increase in performance. While theoretically consistent, LOCI is rarely useful in practice as it essentially investigates univariate associations between the target and a single feature, for which more appropriate methods are available (see above). The group version of LOCI (also included in the WVIM framework) is more useful, where a subset of the features is "left in", chosen, for example, based on domain knowledge about the data.

For statistical inference on LOCO, [Lei et al. \(2018\)](#) suggest an observation-level nonparametric test using the ℓ_1 loss differences on a test set, which is implemented in [xplainfi](#) in a generalized manner that also enables other losses and tests. The corrected t-test approach mentioned in the previous section is also available for LOCO, but it has not been explicitly investigated in this context.

2.4 Methods based on Shapley values

Shapley values originate in cooperative game theory (see [Shapley, 1953](#)) and, in ML, are often used for "local" explanations, i.e., for explaining individual model predictions for selected data instances. But several methods have also been proposed to apply the underlying principle to global FI, including SFIMP (*Shapley Feature IMPortance*) by [Casalicchio et al. \(2019\)](#) and SPVIM (*Shapley Population Variable Importance Measure*) by [Williamson and Feng \(2020\)](#), which is related to the more widely adopted SAGE (*Shapley Additive Global importanceE*) by [Covert et al. \(2020\)](#). [xplainfi](#) implements SAGE, which distributes the overall model loss across the features based on their individual contributions, yielding importance scores that sum to the overall model loss. The SAGE value for feature X_j is defined as the weighted average of its marginal contributions to feature subsets S , also called coalitions, based on a

Table 1: Overview of FI methods and capabilities in `xplainfi` and related R packages and selected Python packages. A checkmark indicates an available feature, and a checkmark in parentheses indicates a feature is not directly offered as such but available in either a limited or indirect fashion. Model importance refers to analyzing a pre-fit model; learner importance refers to refitting models via resampling. Statistical inference denotes the availability of confidence intervals or hypothesis tests for importance estimates.

	xplainfi	vimp	vip	iml	hstats	flashlight	DALEX	fippy	sage
Methods									
PFI	✓		✓	✓	✓	✓	✓	✓	
CFI	✓							✓	
RFI	✓							✓	
LOCO	✓	✓							
WVIM	✓	✓							
mSAGE	✓							✓	✓
cSAGE	✓							✓	(✓)
SPVIM		✓							
Capabilities									
Model importance	✓	✓	✓	✓	✓	✓	✓	✓	✓
Learner importance	✓	✓						(✓)	
Statistical inference	✓	✓						✓	

value function v :

$$\text{SAGE}_j = \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)]. \quad (3)$$

Here, $v(S)$ measures how much better the model performs when the information of the features in S is used for prediction (while features outside S are integrated out) compared to a featureless baseline. *Marginal SAGE* (mSAGE) uses the marginal distribution, resulting in

$$v_m(S) = \mathbb{E} [L(Y, \hat{f}_\phi)] - \mathbb{E} [L(Y, \mathbb{E}_{X_{-S}} [\hat{f}(X_S, X_{-S})])], \quad (4)$$

and *conditional SAGE* (cSAGE) uses the conditional distribution, replacing $\mathbb{E}_{X_{-S}}$ with $\mathbb{E}_{X_{-S}|X_S}$ in Equation (4) to yield $v_c(S)$.

Since exact computation requires evaluating all possible feature coalitions (2^p), implementations must rely on approximations. `xplainfi` implements the permutation estimator described by Covert et al. (2020), where the feature coalitions are built from one of `n_permutations` shuffles of the feature vector. The empty coalition is always evaluated, resulting in a total number of evaluated coalitions of $1 + p \cdot n_permutations$. The variance of SAGE value estimates along the evaluated permutations is also used to define an “early stopping” criterion to avoid excessive computations. Covert et al. (2020) use these variances to construct confidence-like intervals, but we do not include them as statistical inference methods because their coverage is unclear. Additionally, the marginalization step of the estimation requires a subset of size `n_samples` for each data instance, and larger values yield more stable SAGE value estimates but incur additional time and memory costs for large datasets.

3 Related work

Various R packages on CRAN implement one or more FI methods. We compare them along two dimensions: (1) the *scope of FI methods* they provide, and (2) whether they support *model importance* (analyzing a single pre-fit model) or also *learner importance* via resampling and refitting (see Section 2). Table 1 gives a comparative overview.

vip (Greenwell and Boehmke, 2020) requires a pre-fit model and provides model-specific importance extraction, PFI, a variance-based method (“FIRM”), and a Shapley-based measure based on the mean absolute Shapley/SHAP values per feature, i.e., an aggregate of per-observation feature attributions of the model prediction. Unlike loss-based FI methods such as PFI and SAGE, both FIRM and the Shapley-based importance in **vip** summarize importance on the model-output (prediction) scale rather than via changes in loss. It supports a wide range of model classes and offers repeated permutations and optional subsampling of the evaluation data, but does not refit models; accordingly, it targets model importance. Similarly, **iml** (no longer actively developed) operates on pre-fit models

via a Predictor wrapper and provides PFI alongside other IML methods such as feature effects, interaction statistics, and local explanations. **DALEX** likewise wraps pre-fit models via an explainer object; its PFI implementation is provided by the companion package **ingredients**, complemented by local explanations and feature-effect visualizations. **hstats** provides PFI alongside H-statistics for interaction detection, also operating on pre-fit models. **flashlight** wraps pre-fit models in an explainer abstraction and provides model-agnostic PFI, interaction-strength measures based on Friedman's H-statistics for interaction detection, as well as Shapley-based importance via mean absolute (approximate) SHAP feature attributions aggregated across observations. It supports side-by-side comparison of multiple fitted models via the `multiflashlight` function, which can also be used to compare models that were manually refit on different resampling splits. However, it does not orchestrate refitting/resampling or aggregate fold-wise importance into a single learner importance estimate. The **mlr3** ecosystem includes **mlr3filters**, which provides PFI with resampling support within its feature-selection framework, but is designed for feature ranking rather than importance analysis with uncertainty quantification. Generally, PFI is also available in many model-specific implementations (e.g., in **ranger** and **randomForestSRC** for random forests).

To the best of our knowledge, there is no general-purpose, model-agnostic R implementation of CFI or RFI that mirrors the kind of model-wrapping interfaces available for PFI. **cpi** implements Conditional Predictive Impact (CPI), a dedicated conditional-importance testing framework based on knockoff sampling (Watson and Wright, 2021). It is tied to the knockoff construction rather than providing a general conditional-sampling interface for CFI/RFI and focuses on hypothesis testing (p-values and confidence intervals) rather than a broad FI computation framework. **permimp** implements a version of CFI but is restricted to tree-based methods and is therefore not model-agnostic.

vimp takes a different approach: rather than evaluating a pre-fit model, it refits learners for each evaluated feature subset using the **SuperLearner** framework, which automatically ensembles prediction models to approximate the Bayes-optimal predictor (Williamson et al., 2023). This targets population-level importance with valid confidence intervals and hypothesis tests.¹ It implements what it calls "conditional VIM" (equivalent to LOCO for individual FOIs), "marginal VIM" (equivalent to LOCI), and SPVIM (*Shapley Population Variable Importance Measure*; Williamson and Feng (2020)). SPVIM is a Shapley-based method that, like SAGE, distributes the overall model performance across features according to their marginal contributions. Unlike SAGE, it replaces the marginalization or conditional sampling step with model refitting for each feature coalition, which can be slow, but uses Kernel SHAP (Covert and Lee, 2021) to approximate the Shapley values. **vimp** also accepts pre-computed prediction vectors, which, in principle, allow a form of model importance, but with a notably different API compared to the model-wrapping approach of the other packages. It is tightly coupled with the **SuperLearner** framework and offers less flexibility in resampling strategies.

In the Python ecosystem, `scikit-learn` provides tree-based PFI and model-agnostic PFI, both operating on pre-fit estimators. The ELI5 project similarly provides PFI as its only model-agnostic FI method (eli, 2017). `sage` offers the original (marginal) SAGE implementation with no dedicated interface for conditional sampling, but it allows supplying external surrogate models for the same purpose (Covert et al., 2020; Covert and Lee, 2021). The `fippy` package in Python offers the closest overlap with **xplainfi** in terms of perturbation-based methods, providing PFI, CFI, RFI, and both marginal and conditional SAGE using various sampling methods. It also supports simple Wald-type CIs for FI scores from the observation-based losses on test data if the chosen performance measure is decomposable. Overall, `fippy` is mainly centered around explaining pre-fit estimators on fixed evaluation data, but it also offers a (somewhat underdeveloped) `LearnerExplainer` which only supports CFI in combination with subsampling.

In summary, **xplainfi** offers the most comprehensive coverage of model-agnostic FI methods in R, including PFI, CFI, RFI, LOCO, WVIM, and both marginal and conditional SAGE, with flexible systems covering learners, metrics, resampling, and model/learner importance, including tuning and complex ensembles for the latter. Among the compared R packages, only **xplainfi** and **vimp** provide statistical inference methods for their importance estimates, including confidence intervals and hypothesis tests. In the Python ecosystem, `fippy` also provides inference, but limited to the model-importance setting.

4 An introduction to xplainfi

xplainfi's architecture is heavily inspired by the **mlr3** framework and its extension packages, which also form the foundation of the underlying computational infrastructure. This means primarily two things for the user: 1. The package API is based on **R6** classes with the corresponding object-oriented design, using **mlr3** functions and concepts. 2. **xplainfi**'s basic ML capabilities are determined by

¹But note that the **SuperLearner** interface also allows using only a single learner, which can then be analyzed for learner importance with **vimp**.

the **mlr3** ecosystem. We begin with a brief introduction to **mlr3** and then build from simple to more advanced applications of **xplainfi**. For a complete overview, we refer to [Bischl et al. \(2024\)](#), and in particular to [Foss and Kotthoff \(2024\)](#) for a full introduction.

4.1 A brief introduction to mlr3

mlr3 is an R package and an associated ecosystem of extension packages born from methodological and applied ML research. Its core components are abstractions for the building blocks of ML pipelines. Most importantly for our purposes, these include the objects shown in this short application example:

```
library(mlr3learners) # loads mlr3
set.seed(2026)        # for reproducibility
rr <- resample(
  learner = lrn("classif.ranger", num.trees = 100),
  task = tsk("penguins"),
  resampling = rsmpl("cv", folds = 5))
rr$score(msr("classif.ce"))

#>   task_id   learner_id resampling_id iteration classif.ce
#> 1: penguins classif.ranger          cv          1    0.0000
#> 2: penguins classif.ranger          cv          2    0.0290
#> 3: penguins classif.ranger          cv          3    0.0000
#> 4: penguins classif.ranger          cv          4    0.0000
#> 5: penguins classif.ranger          cv          5    0.0294
#> Hidden columns: task, learner, resampling, prediction_test

rr$aggregate(msr("classif.ce"))

#> classif.ce
#>    0.0117
```

Here, we applied the random forest classification learner as implemented in **ranger** with 100 trees to the penguins dataset ([Horst et al., 2020](#)) in a 5-fold cross-validation procedure, evaluated each iteration with the classification error (CE), and finally calculated the average CE across iterations.

A Task (`tsk()`) encapsulates the data and the learning problem (regression, classification, etc.). Built-in tasks for common datasets are available, e.g., `tsk("penguins")`, and custom tasks can be created from any data frame-like object, e.g., `as_task_regr(mtcars, target = "mpg")`. A Learner (`lrn()`) represents an ML algorithm with its hyperparameters. For example, `lrn("regr.ranger", num.trees = 100)` creates a random forest learner via **ranger** using 100 trees. Learners can also be extended into full ML pipelines with preprocessing, feature extraction, or hyperparameter optimization (see [Thomas, 2024](#), [Becker et al. \(2024\)](#), [Binder et al. \(2021\)](#)). A Measure (`msr()`) quantifies prediction performance, e.g., `msr("regr.mse")` for MSE for a regression task. Some measures are decomposable into observation-wise losses, which is relevant for certain inference methods described below. A Resampling (`rsmpl()`) defines train-test splitting strategies, such as holdout, k-fold CV, bootstrapping, or subsampling, e.g., `rsmpl("cv", folds = 5)`.

The general **xplainfi** API relies on these components and applies them to all implemented feature importance methods. For each feature importance method, the user first defines the method object by specifying a Task as the target for the analysis, a Learner for training and predictions, a Resampling strategy, and a Measure for evaluating predictions. The `$compute()` method is then called to perform the actual computational steps required for the individual importance method. Finally, importance scores can be accessed at different levels of aggregation: `$importance()` returns importance values per feature, aggregated across resampling iterations and repetitions (e.g., permutation repetitions in PFI); `$scores()` returns importance values per feature *and* per resampling iteration and repetition, allowing for custom aggregation or visualization; and `$obs_loss()` returns observation-wise loss scores and importance values, if available for the current importance method and Measure.

In the following, we showcase **xplainfi** on an included DGP and refer to [the package website](#) for additional tutorials and descriptions of this and other illustrative simulation settings.

4.2 Example 1: PFI with xplainfi

We start by calculating PFI on a synthetic task with four normally distributed features x_1 through x_4 , two of which are correlated (x_1 and x_2 , $r = 0.8$) and two of which are independent (x_3 and x_4). The

DGP is $y = 2x_1 + x_3 + \varepsilon$, ($\varepsilon \sim \mathcal{N}(0, 0.04)$), generated by the `sim_dgp_correlated()` simulation utility function included in the package, which produces a Task object.

Model importance

The simplest use case is analyzing a single pre-trained model. We train a **ranger** random forest on a holdout split and compute PFI on the corresponding test set:

```
library(xplainfi)
task <- sim_dgp_correlated(n = 5000, r = 0.8)
lrn_ranger <- lrn("regr.ranger")
resampling_ho <- rsmpl("holdout")$instantiate(task)
lrn_ranger$train(task, row_ids = resampling_ho$train_set(1))

pfi_model <- PFI$new(
  task = task, learner = lrn_ranger, measure = msr("regr.mse"),
  resampling = resampling_ho, n_repeats = 10)
pfi_model$compute()
pfi_model$importance()

#>   feature importance
#> 1:      x1    6.757920
#> 2:      x2    0.157254
#> 3:      x3    1.831843
#> 4:      x4    0.000203
```

When a pre-trained learner is passed, **xplainfi** detects this automatically and skips the training step, using the fitted model directly for prediction. The resampling must be instantiated with exactly one test set (i.e., holdout), as there is only one model to evaluate. This yields *model importance*: the importance scores reflect the behavior of this specific model on this specific test set.

Learner importance

To capture importance at the *learner* level, we pass an untrained learner together with a resampling strategy. **xplainfi** then trains a new model in each resampling iteration, and importance scores are aggregated across iterations:

```
pfi <- PFI$new(
  task = task, learner = lrn("regr.ranger"), measure = msr("regr.mse"),
  resampling = rsmpl("cv", folds = 3), n_repeats = 10)
pfi$compute()
pfi$importance()

#>   feature importance
#> 1:      x1    6.5049792
#> 2:      x2    0.1609858
#> 3:      x3    1.8042476
#> 4:      x4   -0.0000154
```

The construction of the PFI object and the computation are separated: calling `$new()` defines the setup, while `$compute()` performs the actual work. To access individual importance scores for each permutation and resampling iteration, the `$scores()` method is available, which stores the corresponding feature, iteration indices, and the associated measure value of the original (baseline) model and the model post perturbation:

```
head(pfi$scores(), 3)

#>   feature iter_rsmpl iter_repeat regr.mse_baseline regr.mse_post importance
#> 1:      x1          1           1           0.0668           6.39           6.33
#> 2:      x1          1           2           0.0668           6.50           6.43
#> 3:      x1          1           3           0.0668           6.80           6.73
```

Using resampling not only captures learner-level variation but also enables variance-corrected inference methods such as the Nadeau-Bengio correction (see Example 3), which require loss differences across resampling iterations and are therefore not available for model importance.

All other components are easily swappable. Here we rerun PFI with XGBoost using the mean absolute error and subsampling instead:

```
pfi <- PFI$new(
  task = task,
  learner = lrn("regr.xgboost", eta = 0.01, nrounds = 1000),
  measure = msr("regr.mae"),
  resampling = rsm("subsampling", repeats = 10),
  n_repeats = 15)
pfi$compute()
pfi$importance()

#>   feature importance
#> 1:      x1    2.101450
#> 2:      x2    0.003902
#> 3:      x3    0.931480
#> 4:      x4    0.000172
```

A learner tuned by cross-validation integrates the same way. Because `auto_tuner()` wraps a learner into an ordinary `mlr3` learner, it can be passed directly to any importance method: its hyperparameters are tuned by an inner cross-validation within each resampling iteration, while importance is evaluated on the outer split. Here, we tune the random forest's `min.node.size` over a small grid with 10 iterations before computing PFI:

```
library(mlr3tuning)
lrn_tuned <- auto_tuner(
  learner = lrn("regr.ranger", min.node.size = to_tune(1, 50)),
  resampling = rsm("cv", folds = 3), measure = msr("regr.mse"),
  tuner = tnr("grid_search", resolution = 10), term_evals = 10)

pfi <- PFI$new(
  task = task, learner = lrn_tuned, measure = msr("regr.mse"),
  resampling = rsm("cv", folds = 3), n_repeats = 10)
pfi$compute()
pfi$importance()

#>   feature importance
#> 1:      x1    6.5429
#> 2:      x2    0.1214
#> 3:      x3    1.7692
#> 4:      x4   -0.0462
```

`xplainfi`'s capabilities are extendable by the `mlr3` ecosystem: any learner from `mlr3learners` or `mlr3extralearners` (Fischer et al., 2025) can be used, including integrated hyperparameter tuning via `to_tune()` tokens or `auto_tuner()` learners, preprocessing and other pipelines via `mlr3pipelines`, or full AutoML systems (Schneider and Becker, 2024; Binder et al., 2024). Furthermore, any metric from `mlr3measures` is available. The trained models and baseline results are retained via `$resample_result` for further analysis.

4.3 Example 2: CFI and conditional samplers

As discussed in Section 2, PFI can produce misleading results when features are correlated. CFI addresses this by sampling from the conditional distribution, which requires a *conditional sampler*. `xplainfi` provides a modular abstraction for this. Several conditional samplers are available, but here we focus on two: The `ConditionalGaussianSampler` assumes multivariate normality and is fast but limited to numeric features. The `ConditionalARFSampler` uses adversarial random forests (ARF) and handles mixed data, but is computationally more expensive (Watson et al., 2023).

Samplers are instantiated on a given task once, after which one can draw one or more observations, which CFI and related methods handle internally. Each conditional sampler allows the specification of an arbitrary conditioning set of features for sampling.

We continue with the correlated DGP from Example 1, where x_1 and x_2 are correlated ($r = 0.8$), but only x_1 affects the target. Using a Gaussian conditional sampler, we compute CFI and also request quantiles calculated across resampling iterations:

```
cfi <- CFI$new(
  task = task, learner = lrn("regr.ranger"), measure = msr("regr.mse"),
  resampling = rsmpl("subsampling", repeats = 5),
  sampler = ConditionalGaussianSampler$new(task),
  n_repeats = 10)
cfi$compute()
cfi$importance(ci_method = "quantile", alternative = "two.sided")

#>   feature importance conf_lower conf_upper
#> 1:      x1    2.795426    2.776069    2.831263
#> 2:      x2    0.000648   -0.001326    0.002540
#> 3:      x3    1.815376    1.799187    1.831372
#> 4:      x4   -0.000178   -0.000512    0.000496
```

Compared to PFI, CFI identifies that x_2 is not associated with y conditional on the other features. The sampler argument distinguishes CFI from PFI: while PFI uses marginal permutation, CFI requires a conditional sampler. The "quantile" method shown above provides empirical quantiles from the distribution of importance scores across resampling iterations, which help to gauge the stability of the estimates. Note that via "alternative", we explicitly requested the default of "two-sided" intervals, by analogy with two- or one-sided hypothesis tests, where "alternative = 'greater'" would have given us only the 5% quantile as lower bound.

4.4 Example 3: Inference

For principled inference beyond empirical quantiles, `xplainfi` supports two main approaches, depending on the chosen importance method.

Nadeau-Bengio correction for PFI

[Molnar et al. \(2023\)](#) recommend variance correction based on [Nadeau and Bengio \(2003\)](#) when using PFI with subsampling or bootstrapping. The correction accounts for dependence between resampling iterations that share training observations, yielding confidence intervals with improved (though still imperfect) coverage. The recommended setup uses approximately 15 subsampling iterations. Since our earlier PFI example already uses subsampling, we can request corrected confidence intervals via `ci_method = "nadeau_bengio"`:

```
head(pfi$importance(ci_method = "nadeau_bengio"), 3)

#>   feature importance      se statistic p.value conf_lower conf_upper
#> 1:      x1      6.543 0.12415    52.7 0.000360    6.0087    7.077
#> 2:      x2      0.121 0.00532    22.8 0.001917    0.0985    0.144
#> 3:      x3      1.769 0.05474    32.3 0.000956    1.5337    2.005
```

The output includes standard errors and adjusted confidence intervals. Note that this approach assumes normally distributed importance scores and was primarily evaluated for PFI; its use with other methods is experimental. Figure 1 compares the corrected confidence intervals with the empirical 95% quantiles and also unadjusted confidence intervals. We note that the latter are also available via `ci_method = "raw"` for comparison, but they are not valid for inference.

CPI for conditional importance

For CFI, inference is available through the CPI framework when using knockoff-based sampling, which leverages observation-wise losses to perform hypothesis tests for feature importance ([Watson and Wright, 2021](#)). Using a knockoff sampler with CFI enables `ci_method = "cpi"`, for example in conjunction with a t-test:

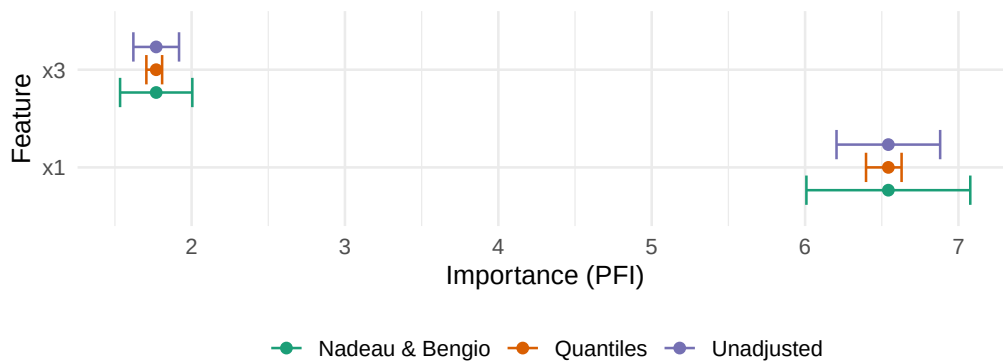


Figure 1: Comparison of uncertainty quantification methods using PFI on the correlated task (omitting noise features). Empirical 95% quantiles fall between the very narrow unadjusted confidence intervals and the wider Nadeau-Bengio-corrected intervals, which show the uncertainty masked by the unadjusted method.

```
cfi_knockoff <- CFI$new(
  task = task, learner = lrn("regr.ranger"), measure = msr("regr.mse"),
  sampler = KnockoffGaussianSampler$new(task),
  resampling = rsmpl("holdout"), n_repeats = 1)
cfi_knockoff$compute()
cfi_knockoff$importance(
  ci_method = "cpi", alternative = "greater",
  p_adjust = "BH", test = "t")
```

#>	feature	importance	se	statistic	p.value	conf_lower	conf_upper
#> 1:	x1	3.013539	0.102100	29.516	4.52e-154	2.84551	Inf
#> 2:	x2	0.000550	0.002232	0.246	5.37e-01	-0.00312	Inf
#> 3:	x3	1.760609	0.060914	28.903	3.14e-149	1.66036	Inf
#> 4:	x4	-0.000492	0.000569	-0.865	8.06e-01	-0.00143	Inf

This CPI test provides p-values and one-sided confidence bounds for each feature. Here, we use the `p_adjust` argument to use R's `p.adjust()` function to correct for the FDR with the Benjamini-Hochberg procedure. We note that this only corrects the p-values, but not the confidence bounds, which would only be adjusted for `p_adjust = "bonferroni"`. Alternatively to the knockoff approach, `ConditionalARFSampler` can be used in place of knockoffs, enabling CPI-style inference for mixed data types as proposed by Blesch et al. (2025), though at a higher computational cost. Note that CPI was proposed in combination with a fixed model and a test set, whereas `xplainfi` also allows its use with cross-validation. Because the CPI test operates on observation-wise losses, the non-overlapping test sets from the resampling folds are pooled into a single set of observation-wise losses on which the test is computed (i.e., micro-averaging rather than averaging per-fold results).

4.5 Example 4: LOCO and WVIM

LOCO is implemented as a special case of the more general WVIM framework as described in Section 2. We apply it to the simulated task from Example 1 using a single refit iteration and show the same Bonferroni-adjusted p-values and confidence intervals based on the Wilcoxon test proposed by Lei et al. (2018):

```
loco <- LOCO$new(
  task = task, learner = lrn("regr.ranger"),
  measure = msr("regr.mse"), resampling = rsmpl("holdout"))
loco$compute()
loco$importance(
  ci_method = "lei", test = "wilcox",
  p_adjust = "bonferroni")[, c(1:2, 5:7)]
```

#>	feature	importance	p.value	conf_lower	conf_upper
#> 1:	x1	0.74379	7.22e-242	1.03073	1.26974
#> 2:	x2	0.00203	5.66e-09	0.00293	0.00758

```
#> 3:      x3      0.47730 9.08e-222      0.68221      0.83865
#> 4:      x4      0.00477 4.64e-46      0.00767      0.01195
```

WVIM supports arbitrary feature groups via the `groups` argument, which accepts a named list specifying features belonging to each group (i.e., LOGO):

```
groups <- list(correlated = c("x1", "x2"), independent = c("x3", "x4"))
wvim <- WVIM$new(
  task = task, learner = lrn("regr.ranger"),
  groups = groups, direction = "leave-out",
  measure = msr("regr.mse"), resampling = rsmp("holdout"))
wvim$compute()
wvim$importance()

#>      feature importance
#> 1: correlated      4.40
#> 2: independent      1.11
```

Here, the "correlated" group contains features `x1` and `x2`, which are removed together. WVIM also supports a "leave-in" direction, which trains models with *only* the specified features and compares against a featureless baseline (i.e., LOCI or "LOGI"). The `groups` argument is also available for PFI, CFI, and RFI, where the specified groups of features are then always perturbed at once.

4.6 Conditional samplers

As described in Section 2, conditional methods like CFI and cSAGE require a mechanism to sample from the conditional distribution $P(X_j|X_{-j})$. [xplainfi](#) provides a modular sampler architecture that allows different sampling strategies to be instantiated and passed to the importance method. All samplers inherit from the `FeatureSampler` base class and share a common interface: Instantiate a sampler on a `Task`, then use the `$sample()` method to draw new values for specified features conditional on the remaining ones or a subset of features specified as `conditioning_set`.

We demonstrate the ARF sampler on the penguins task, which contains both numeric and categorical features. After instantiation, we sample new values for the `body_mass` feature for a few observations, conditional on all other features (including the categorical `island`):

```
task_penguins <- tsk("penguins")
sampler_arf <- ConditionalARFSampler$new(task_penguins)
# Original values for comparison
task_penguins$data(
  rows = c(1, 20, 40),
  cols = c("island", "bill_length", "body_mass"))

#>      island bill_length body_mass
#> 1: Torgersen      39.1      3750
#> 2: Torgersen      46.0      4200
#> 3: Dream         39.8      4650

# Sample new body_mass values conditional on all other features
sampler_arf$sample(feature = "body_mass", row_ids = c(1, 20, 40))[, c(6, 3, 4)]

#>      island bill_length body_mass
#> 1: Torgersen      39.1      3774
#> 2: Torgersen      46.0      4467
#> 3: Dream         39.8      3175
```

The sampler produces new values for `body_mass` drawn from the estimated conditional distribution $P(\text{body_mass}|X_{-\text{body_mass}})$. These sampled values replace the original feature when computing CFI or cSAGE, allowing importance to be assessed while preserving dependencies with other features.

The choice of sampler involves trade-offs between flexibility and computational cost. The ARF sampler handles mixed data types without making distributional assumptions but requires fitting an adversarial random forest, which increases computational time and memory usage.

5 Benchmark experiments

To evaluate **xplainfi**, we performed benchmark experiments in two categories: **Importance results**: Since **xplainfi** re-implements methods available in other packages, we verify that it produces equivalent importance scores across shared methods and DGPs, establishing that the implementations are faithful and comparable across packages. **Runtime**: We assess whether **xplainfi**'s implementations are competitive in terms of computational cost compared to existing single-method packages, across varying task dimensionalities and method parameters.

We compare against packages that implement the same methods in a model-importance setting, as this is the common denominator across implementations (see Section 3). For PFI, we compare with **iml** and **vip**. For marginal SAGE, we compare with **fippy** and **sage**, the latter implementing kernel SAGE (Covert and Lee, 2021). For CFI and conditional SAGE, we compare with **fippy**, using a Gaussian conditional sampler as the lowest common denominator of available sampling options. **vimp** is not included as it only shares the refitting-based methods (LOCO, WVIM) with **xplainfi**, and differences in supported metrics, learner frameworks, and resampling strategies make a fair apples-to-apples comparison difficult. The MSE was used as the evaluation metric across all methods, except for **vip**, which only supports the root mean squared error. Since RMSE is a monotone transformation of MSE, feature rankings are preserved but scaled importance magnitudes differ. Both benchmarks are defined in R using **batchtools** (Lang et al., 2017) for cluster-based parallelization, and **reticulate** is used to run Python implementations.

Each method is evaluated using one of four learners to account for variability in learner capabilities: a linear model (**stats** / **scikit-learn** (Pedregosa et al., 2011)), a random forest (**ranger** / **scikit-learn**, 500 trees), XGBoost (Chen and Guestrin, 2016) (1000 rounds, early stopping after 50, $\eta = 0.1$), and an MLP (**mlr3torch** / **scikit-learn**, 1 hidden layer, 20 neurons, 500 epochs with early stopping). Of these, XGBoost is the only learner with an identical underlying implementation across R and Python. The linear model is expected to yield approximately identical results in either language. Learner configurations were chosen for simplicity and robustness rather than predictive performance, i.e., learners were not tuned beyond built-in regularization mechanisms.

5.1 Importance benchmark

We compare importance scores by scaling values for a given method and DGP to the unit interval to examine relative magnitudes. A rank-based comparison is available in the online supplement.

We compare five FI methods across one to four different implementations. Table 2 lists these methods and packages, alongside the parameters used for each method where applicable. To obtain reliable FI estimates, we used 50 repetitions for PFI and CFI, and similarly a high number of permutations for mSAGE and cSAGE, while enabling the “early stopping” options in both **xplainfi**'s and **fippy**'s implementations to avoid high computational cost at diminishing returns. Since we prioritize comparability, we used Gaussian conditional sampling for the conditional methods CFI and cSAGE throughout the experiment, as this is the only sampling mechanism implemented by both **xplainfi** and **fippy**. Both packages offer additional sampling mechanisms with greater flexibility, but would have introduced more variability into the comparison. The tasks selected for this experiment consist of simulation settings with DGPs designed to showcase differences and similarities among feature importance methods; a full listing is provided in the online supplement. Here, we focus on the “correlated” DGP generated by `xplainfi::sim_dgp_correlated()` as introduced in the previous section ($Y = 2X_1 + X_3 + \varepsilon$, with $\text{cor}(X_1, X_2) \in \{0.25, 0.75\}$, $p = 4$), and the “bike sharing” dataset introduced by Fanaee-T and Gama (2014) ($p = 12$), which is frequently used as an example for FI methods (e.g., Covert et al., 2020; Blesch et al., 2025). The bike sharing dataset's categorical features were converted to numeric values, as this was required to ensure comparability between **xplainfi** and **fippy** via the Gaussian conditional sampler. For simulated data, $n = 5000$ samples were generated and 2/3 of the observations were used for learner training and 1/3 for feature importance calculation as a test set. The bike sharing dataset was resampled in every replication with a different train-test split. The train and test sets were created consistently across the different implementations, i.e., the linear model for **xplainfi** was trained on the same data as the one for **vip** or **fippy** within a replication. Importance scores shown are aggregated from 25 replications. For additional details on the simulated settings, see the online supplement or **xplainfi**'s online documentation.

Due to the large number of factors in the experiment, we present a subset of results here and refer to the online supplement for a complete overview. We focus on the linear and boosting learners since they are the most comparable across R and Python.

Figure 2 shows almost identical scaled importance scores across all implementing packages for PFI and CFI. The exception is **vip**, which produces the same feature ranking but with differing importance scores, most likely due to the different evaluation metric. The feature ranking is equivalent between

Table 2: Methods, implementations, and parameters for both benchmarks. Parameter names correspond to `xplainfi`’s API and are set equivalently in other implementations. For the importance benchmark, early stopping was enabled for `mSAGE` and `cSAGE` in `xplainfi` and `fippy`; for the runtime benchmark, it was disabled to ensure identical workloads.

Method	Packages	Parameters
PFI	<code>xplainfi</code> , <code>fippy</code> , <code>iml</code> , <code>vip</code>	Importance: <code>n_repeats</code> = 50 Runtime: <code>n_repeats</code> = {1, 50}
CFI	<code>xplainfi</code> , <code>fippy</code>	Importance: <code>n_repeats</code> = 50, <code>sampler</code> : Gaussian Runtime: <code>n_repeats</code> = {1, 50}, <code>sampler</code> : Gaussian
mSAGE	<code>xplainfi</code> , <code>fippy</code>	Importance: <code>n_permutations</code> = 100, <code>n_samples</code> = 100, <code>min_permutations</code> = 20 Runtime: <code>n_permutations</code> = {10, 20, 50}, <code>n_samples</code> = {10, 50, 100}
mSAGE	<code>sage</code>	Importance: <code>n_samples</code> = 100 Runtime: <code>n_samples</code> = {10, 50, 100}
cSAGE	<code>xplainfi</code> , <code>fippy</code>	Importance: <code>n_permutations</code> = 100, <code>n_samples</code> = 100, <code>min_permutations</code> = 20, <code>sampler</code> : Gaussian Runtime: <code>n_permutations</code> = {10, 20, 50}, <code>n_samples</code> = {10, 50, 100}, <code>sampler</code> : Gaussian

the two learner types. For `mSAGE` and `cSAGE`, similarly close agreement is visible between `xplainfi` and `fippy`, while `sage`’s kernel `SAGE` shows scores close to the other implementations with a slightly larger variance.

Next, we consider the bike sharing task. Figure 3 shows PFI and CFI using the linear model and boosting learner. For PFI, we see strong agreement between the methods similar to the previous setting. For CFI, `xplainfi` and `fippy` produce different (scaled) importance scores for the year and working_day features. This is most likely explained by the Gaussian conditional sampler being implemented slightly differently in both packages, combined with its use here on effectively categorical features encoded as integers, which is not the most appropriate choice. A similar pattern is visible in Figure 4 for `cSAGE` with the same underlying cause but good agreement otherwise. For `mSAGE`, `sage`’s importance scores produce a different pattern which does not appear to agree with the other methods. The linear model’s importance scores additionally show comparatively large variance (e.g., for `apparent_temperature`), which reflects multicollinearity among the bike sharing features: `temperature` and `apparent_temperature` are strongly correlated. The coefficients of this unregularized, main-effects-only learner are therefore unstable across train-test splits, and the loss-based importance inherits this instability. Scores for the remaining simulation settings and learners are available in the online supplement. In all settings, `xplainfi`’s results agree with the reference implementations apart from minor deviations that did not affect the overall ranking, as also shown by the rank-based and paired-difference analyses in the supplement.

5.2 Runtime benchmark

Methods were run exclusively using linear models to isolate the computational cost of the FI method itself from that of the learner (see Table 2 for parameter ranges). We used only the peak task from `mlbench`, which provides a regression problem with user-definable sample and feature sizes. Experiments were run for 25 replications on the Intel Xeon Platinum 8380 compute nodes of the Leibniz Supercomputing Centre.

Figure 5 shows median runtimes with 25% and 75% quantiles for $n = 5000$ with varying p . For PFI, `iml` is clearly the slowest implementation after `fippy`. `vip` shows only slightly slower times than `xplainfi`. For `mSAGE`, `sage` notably becomes faster with more features, likely because the kernel `SAGE` implementation converges faster in that case. Across all methods, `fippy` is notably slower than `xplainfi`. Overall, `xplainfi` was consistently faster than or comparable to the reference implementations under close-to-equal settings. Additional results for all methods and parameter configurations are available in the online supplement.

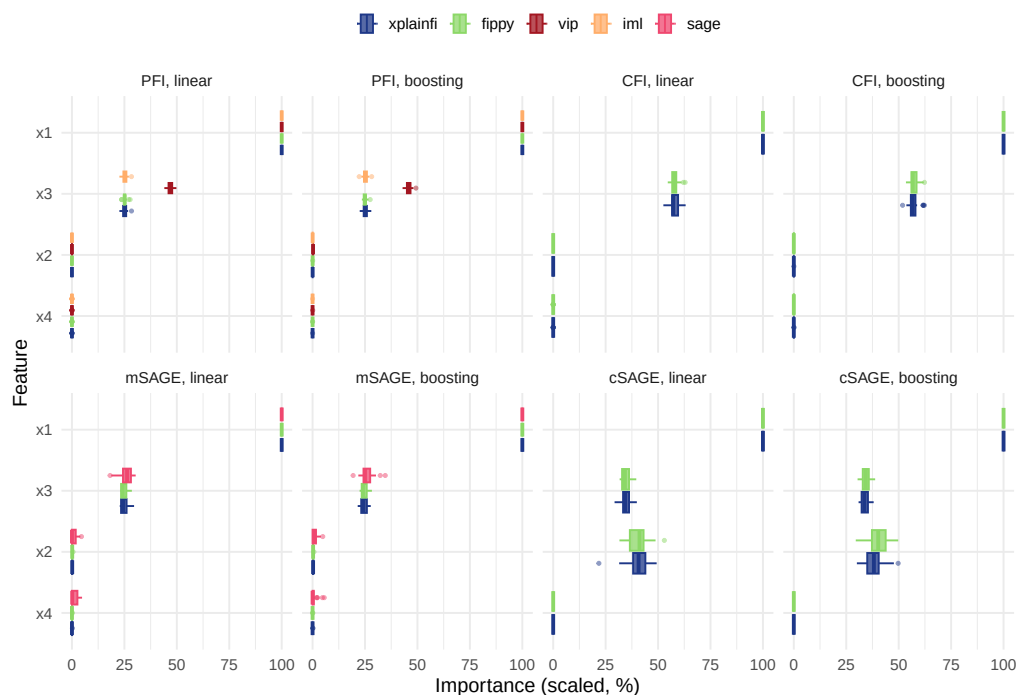


Figure 2: Importance scores (scaled to percentages) for PFI, CFI, mSAGE, and cSAGE across implementations on the correlated simulation setting with $r = 0.75$, based on either the linear model or the boosting learner.

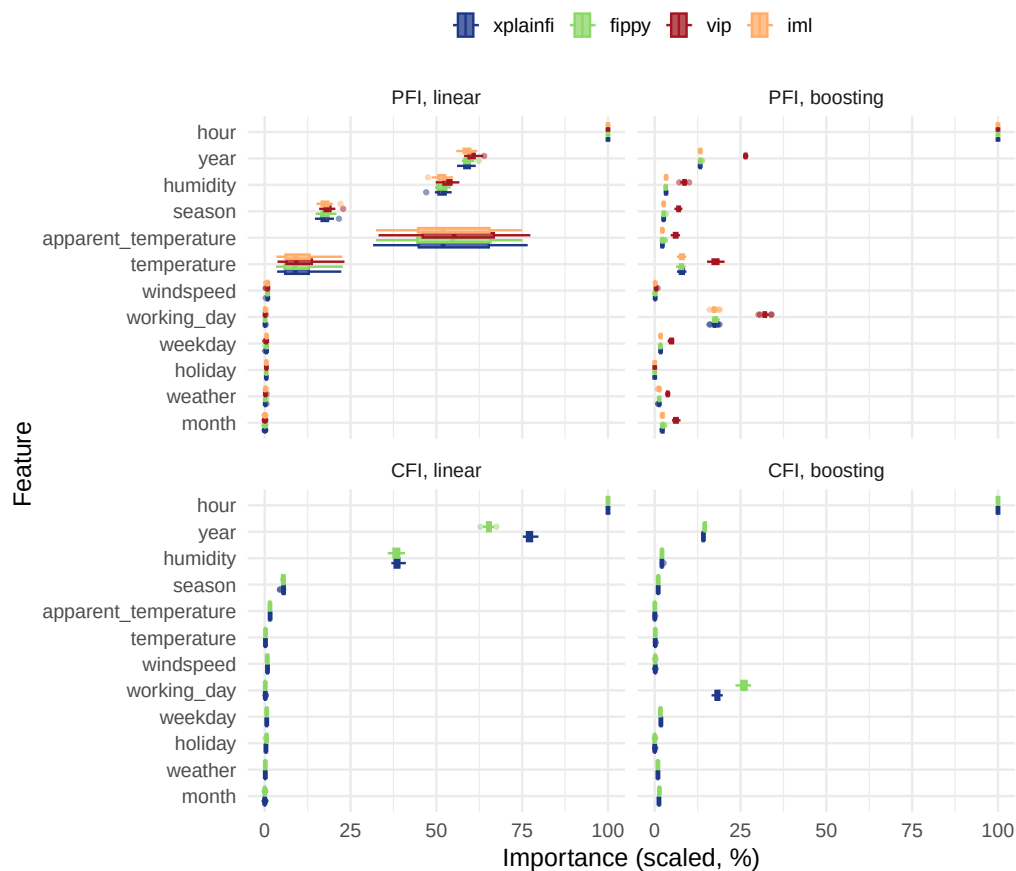


Figure 3: Importance scores (scaled to percentages) for PFI and CFI across implementations on the bike sharing dataset, based on either the linear model or the boosting learner.

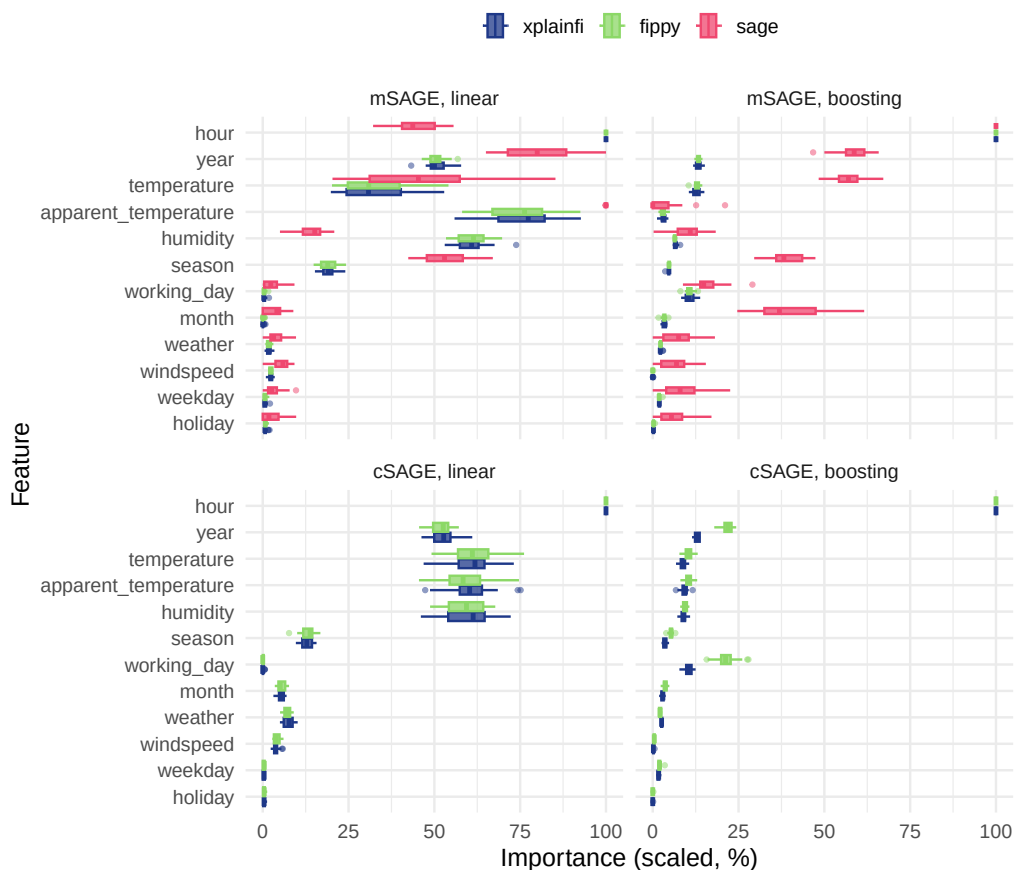


Figure 4: Importance scores (scaled to percentages) for mSAGE and cSAGE across implementations on the bike sharing dataset, based on either the linear model or the boosting learner.

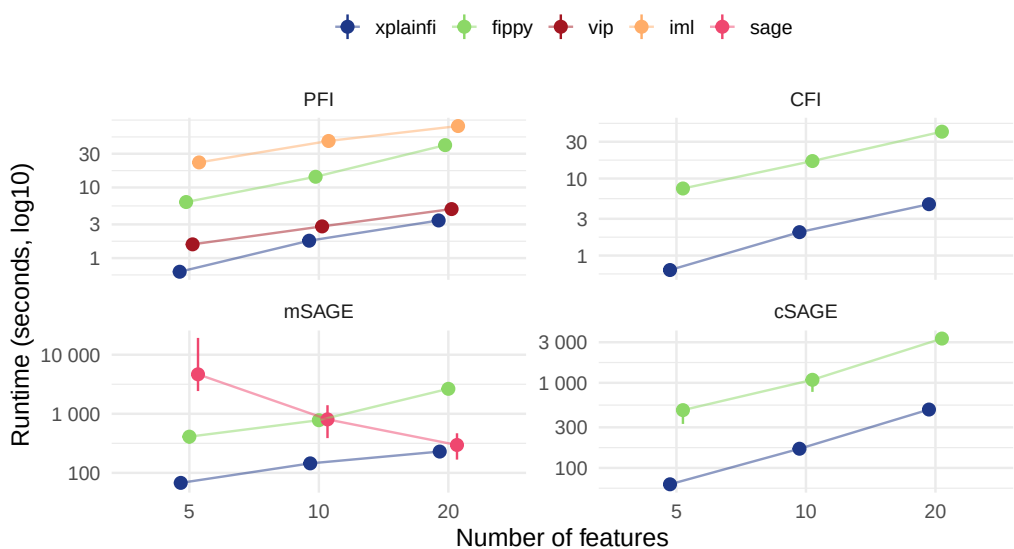


Figure 5: Median runtime in seconds with 25% and 75% quantiles for PFI and CFI with $n_{\text{repeats}} = 50$ and mSAGE and cSAGE with $n_{\text{samples}} = 100$ and $n_{\text{permutations}} = 100$ across implementations on the peak simulation setting with 5, 10, and 20 features and 5000 samples using a linear model across 25 replications.

6 Discussion and conclusion

We introduced `xplainfi`, a package that provides a comprehensive suite of global, loss-based FI methods within a unified framework built on the `mlr3` ecosystem, by implementing methods previously unavailable or scattered across packages, including conditional feature importance (CFI), relative feature importance (RFI), and both marginal and conditional SAGE. This allows `xplainfi` to focus primarily on FI methods without reimplementing common ML building blocks such as model interfaces, resampling, and performance measures. However, this also means `xplainfi` is not compatible with related frameworks such as `tidymodels` or arbitrary “unwrapped” models and learner implementations. We accept this compromise, as we could not otherwise maintain or expand the current feature set. Our benchmark experiments demonstrate that `xplainfi` produces importance scores consistent with reference implementations across multiple DGPs and learner types, while offering competitive runtime performance.

Choosing the “best” method depends on the research question, the type of association of interest (marginal vs. conditional), and the available computational budget, which can affect the choice between perturbation-based, refit-based, and Shapley-based methods. For detailed guidance on underlying estimands and interpretation, we refer to [Ewald et al. \(2024\)](#).

The modular feature sampler architecture is unique, enabling `xplainfi` to perform conditional FI analysis on continuous and mixed data alike via flexible samplers. This includes the ability to specify arbitrary conditioning sets for each sampler in the Conditional family, which enables conditional SAGE and RFI. Interest in conditional methods has been growing, and future work may benefit from the building blocks available with `xplainfi`. However, these methods come with trade-offs: the ARF sampler, while flexible, incurs higher computational cost and memory usage than Gaussian sampling, and SAGE methods can be slow due to the many feature coalitions that must be evaluated. Subsequent releases have reduced the computational cost of these methods, and a kernel-based SAGE estimator is in development.

The package could and should be extended to include additional confidence interval methods, or more generally, uncertainty quantification methods for FI. Although this is very desirable from a practical perspective, not enough (established) techniques and coverage studies exist, and we think exposing users to unvalidated techniques is not appropriate; such dedicated studies are out of scope for the current paper.

The package currently offers no visualization options, as we focused on the computational aspects. Visualizations for reports and publications often have specific requirements and are customized accordingly. Because we provide well-structured container types, the FI values (and other results) can be readily plotted using, e.g., `ggplot2`.

7 Acknowledgements

The authors gratefully acknowledge the computational and data resources provided by the Leibniz Supercomputing Centre (www.lrz.de).

Lukas Burk is supported by the Federal Ministry of Research, Technology and Space (BMFTR), grant number 01EQ2409E.

Marvin N. Wright is supported by the German Research Foundation (DFG), grant numbers 437611051 and 459360854, and the Federal Ministry of Research, Technology and Space (BMFTR), grant number 01EQ2409E.

Bernd Bischl is supported by the German Research Foundation (DFG), grant number 460135501.

References

- Eli5: Debug machine learning classifiers and explain their predictions, 2017. URL <https://github.com/eli5-org/eli5>. [p363]
- Q. Au, J. Herbinger, C. Stachl, B. Bischl, and G. Casalicchio. Grouped feature importance and combined features effect plot. *Data Mining and Knowledge Discovery*, 36(4):1401–1450, 2022. doi: 10.1007/s10618-022-00840-5. [p361]
- M. Becker, L. Schneider, and S. Fischer. Hyperparameter optimization. In B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors, *Applied Machine Learning Using mlr3 in R*. CRC Press, 2024. URL https://mlr3book.mlr-org.com/hyperparameter_optimization.html. [p364]

- Y. Benjamini and Y. Hochberg. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1): 289–300, 1995. ISSN 0035-9246. doi: 10.1111/j.2517-6161.1995.tb02031.x. [p360]
- Y. Benjamini and D. Yekutieli. The control of the false discovery rate in multiple testing under dependency. *The Annals of Statistics*, 29(4):1165–1188, 2001. ISSN 0090-5364, 2168-8966. doi: 10.1214/aos/1013699998. [p360]
- M. Binder, F. Pfisterer, M. Lang, L. Schneider, L. Kotthoff, and B. Bischl. Mlr3pipelines - Flexible Machine Learning Pipelines in R. *Journal of Machine Learning Research*, 22(184):1–7, 2021. ISSN 1533-7928. URL <http://jmlr.org/papers/v22/21-0281.html>. [p358, 364]
- M. Binder, F. Pfisterer, M. Becker, and M. N. Wright. Non-sequential pipelines and tuning. In B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors, *Applied Machine Learning Using mlr3* in R. CRC Press, 2024. URL https://mlr3book.mlr-org.com/non-sequential_pipelines_and_tuning.html. [p366]
- B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors. *Applied Machine Learning Using mlr3* in R. CRC Press, 2024. ISBN 978-1-032-50754-5. URL <https://mlr3book.mlr-org.com>. [p364]
- K. Blesch, D. S. Watson, and M. N. Wright. Conditional feature importance for mixed data. *AStA Advances in Statistical Analysis*, 108(2):259–278, 2024. ISSN 1863-818X. doi: 10.1007/s10182-023-00477-9. [p358, 361]
- K. Blesch, N. Koenen, J. Kapar, P. Golchian, L. Burk, M. Loecher, and M. N. Wright. Conditional Feature Importance with Generative Modeling Using Adversarial Random Forests. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(15):15596–15604, 2025. ISSN 2374-3468. doi: 10.1609/aaai.v39i15.33712. [p361, 368, 370]
- A. Bommert, X. Sun, B. Bischl, J. Rahnenführer, and M. Lang. Benchmark for filter methods for feature selection in high-dimensional classification data. *Computational Statistics & Data Analysis*, 143:106839, 2020. ISSN 0167-9473. doi: 10.1016/j.csda.2019.106839. [p359]
- L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. ISSN 1573-0565. doi: 10.1023/A:1010933404324. [p359, 360]
- E. Candès, Y. Fan, L. Janson, and J. Lv. Panning for Gold: ‘Model-X’ Knockoffs for High Dimensional Controlled Variable Selection. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 80(3):551–577, 2018. ISSN 1369-7412. doi: 10.1111/rssb.12265. [p359, 361]
- G. Casalicchio, C. Molnar, and B. Bischl. Visualizing the Feature Importance for Black Box Models. In M. Berlingerio, F. Bonchi, T. Gärtner, N. Hurley, and G. Ifrim, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 655–670, Cham, 2019. Springer International Publishing. ISBN 978-3-030-10925-7. doi: 10.1007/978-3-030-10925-7_40. [p361]
- T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pages 785–794, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 978-1-4503-4232-2. doi: 10.1145/2939672.2939785. [p370]
- T. Chiaburu, F. Haußer, and F. Bießmann. Uncertainty in xai: Human perception and modeling approaches. *Machine Learning and Knowledge Extraction*, 6(2):1170–1192, 2024. doi: 10.3390/make6020055. [p359]
- I. Covert and S.-I. Lee. Improving KernelSHAP: Practical Shapley Value Estimation Using Linear Regression. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, pages 3457–3465. PMLR, 2021. URL <https://proceedings.mlr.press/v130/covert21a.html>. [p363, 370]
- I. Covert, S. M. Lundberg, and S.-I. Lee. Understanding Global Feature Contributions With Additive Importance Measures. In *Advances in Neural Information Processing Systems*, volume 33, pages 17212–17223. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/c7bf0b7c1a86d5eb3be2c722cf746-Abstract.html>. [p359, 361, 362, 363, 370]
- D. Debeer and C. Strobl. Conditional permutation importance revisited. *BMC Bioinformatics*, 21(1):307, 2020. ISSN 1471-2105. doi: 10.1186/s12859-020-03622-2. [p358, 359]
- N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola. AutoGluon-tabular: Robust and accurate AutoML for structured data. 2020. URL <https://arxiv.org/abs/2003.06505>. [p360]

- F. K. Ewald, L. Bothmann, M. N. Wright, B. Bischl, G. Casalicchio, and G. König. A Guide to Feature Importance Methods for Scientific Inference. In L. Longo, S. Lapuschkin, and C. Seifert, editors, *Explainable Artificial Intelligence*, pages 440–464, Cham, 2024. Springer Nature Switzerland. ISBN 978-3-031-63797-1. doi: 10.1007/978-3-031-63797-1_22. [p358, 359, 360, 361, 374]
- H. Fanaee-T and J. Gama. Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, 2(2):113–127, 2014. ISSN 2192-6360. doi: 10.1007/s13748-013-0040-3. [p370]
- S. Fischer, J. Zobolas, R. Sonabend, M. Becker, M. Lang, M. Binder, L. Schneider, L. Burk, P. Schratz, B. C. Jaeger, S. A. Lauer, L. A. Kapsner, M. Mücke, Z. Wang, D. Pulatov, K. Ganz, H. Funk, L. Harutyunyan, P. Camilleri, P. Kopper, A. Bender, B. Zhou, N. German, L. Koers, A. Nazarova, and B. Bischl. Mlr3extralearners: Expanding the mlr3 Ecosystem with Community-Driven Learner Integration. *Journal of Open Source Software*, 10(115):8331, 2025. ISSN 2475-9066. doi: 10.21105/joss.08331. [p366]
- A. Fisher, C. Rudin, and F. Dominici. All Models are Wrong, but Many are Useful: Learning a Variable’s Importance by Studying an Entire Class of Prediction Models Simultaneously. *Journal of machine learning research : JMLR*, 20:177, 2019. ISSN 1532-4435. URL <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8323609/>. [p358]
- N. Foss and L. Kotthoff. Data and basic modeling. In B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors, *Applied Machine Learning Using mlr3 in R*. CRC Press, 2024. URL https://mlr3book.mlr-org.com/data_and_basic_modeling.html. [p364]
- B. M. Greenwell and B. C. Boehmke. Variable importance plots—an introduction to the vip package. *The R Journal*, 12(1):343–366, 2020. doi: 10.32614/RJ-2020-013. [p362]
- R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi. A survey of methods for explaining black box models. *ACM computing surveys (CSUR)*, 51(5):1–42, 2018. doi: 10.1145/3236009. [p358]
- I. Guyon and A. Elisseeff. An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3(Mar):1157–1182, 2003. ISSN 1533-7928. URL <https://www.jmlr.org/papers/v3/guyon03a.html>. [p358]
- S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. ISSN 0303-6898. URL <https://www.jstor.org/stable/4615733>. [p360]
- G. Hooker, L. Mentch, and S. Zhou. Unrestricted permutation forces extrapolation: Variable importance requires at least one more model, or there is no free variable importance. *Statistics and Computing*, 31(6):82, Oct. 2021. ISSN 1573-1375. doi: 10.1007/s11222-021-10057-z. [p358, 360]
- A. M. Horst, A. P. Hill, and K. B. Gorman. *Palmerpenguins: Palmer Archipelago (Antarctica) Penguin Data*, 2020. [p364]
- T. Hothorn, K. Hornik, and A. Zeileis. Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics*, 15(3):651–674, 2006. ISSN 1061-8600, 1537-2715. doi: 10.1198/106186006X133933. [p359]
- G. König, C. Molnar, B. Bischl, and M. Grosse-Wentrup. Relative Feature Importance. pages 9318–9325, 2021. doi: 10.1109/ICPR48806.2021.9413090. [p359, 360]
- M. Lang, B. Bischl, and D. Surmann. Batchtools: Tools for R to work on batch systems. *Journal of Open Source Software*, 2(10):135, Feb. 2017. ISSN 2475-9066. doi: 10.21105/joss.00135. [p370]
- M. Lang, M. Binder, J. Richter, P. Schratz, F. Pfisterer, S. Coors, Q. Au, G. Casalicchio, L. Kotthoff, and B. Bischl. Mlr3: A modern object-oriented machine learning framework in R. *Journal of Open Source Software*, 4(44):1903, 2019. ISSN 2475-9066. doi: 10.21105/joss.01903. [p358]
- J. Lei, G. Max, R. Alessandro, T. Ryan J., and L. Wasserman. Distribution-Free Predictive Inference for Regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018. ISSN 0162-1459. doi: 10.1080/01621459.2017.1307116. [p359, 361, 368]
- J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu. Feature Selection: A Data Perspective. *ACM Computing Surveys*, 50(6):1–45, Nov. 2018. ISSN 0360-0300, 1557-7341. doi: 10.1145/3136625. [p359]
- C. Molnar. *Interpretable machine learning*. Lulu. com, 2020. URL <https://christophm.github.io/interpretable-ml-book/>. [p358]

- C. Molnar, T. Freiesleben, G. König, J. Herbringer, T. Reisinger, G. Casalicchio, M. N. Wright, and B. Bischl. Relating the Partial Dependence Plot and Permutation Feature Importance to the Data Generating Process. In L. Longo, editor, *Explainable Artificial Intelligence*, pages 456–479, Cham, 2023. Springer Nature Switzerland. ISBN 978-3-031-44064-9. doi: 10.1007/978-3-031-44064-9_24. [p359, 360, 367]
- W. J. Murdoch, C. Singh, K. Kumbier, R. Abbasi-Asl, and B. Yu. Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences*, 116(44):22071–22080, 2019. doi: 10.1073/pnas.1900654116. [p358]
- C. Nadeau and Y. Bengio. Inference for the Generalization Error. *Machine Learning*, 52(3):239–281, 2003. ISSN 1573-0565. doi: 10.1023/A:1024068626366. [p359, 360, 367]
- K. K. Nicodemus, J. D. Malley, C. Strobl, and A. Ziegler. The behaviour of random forest permutation-based variable importance measures under predictor correlation. *BMC Bioinformatics*, 11(1):110, 2010. ISSN 1471-2105. doi: 10.1186/1471-2105-11-110. [p358]
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>. [p370]
- A. Redelmeier, M. Jullum, and K. Aas. Explaining Predictive Models with Mixed Features Using Shapley Values and Conditional Inference Trees. In A. Holzinger, P. Kieseberg, A. M. Tjoa, and E. Weippl, editors, *Machine Learning and Knowledge Extraction*, pages 117–137, Cham, 2020. Springer International Publishing. ISBN 978-3-030-57321-8. doi: 10.1007/978-3-030-57321-8_7. [p358, 361]
- B. Rozemberczki, L. Watson, P. Bayer, H.-T. Yang, O. Kiss, S. Nilsson, and R. Sarkar. The Shapley Value in Machine Learning. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, pages 5572–5579, Vienna, Austria, 2022. International Joint Conferences on Artificial Intelligence Organization. ISBN 978-1-956792-00-3. doi: 10.24963/ijcai.2022/778. [p358]
- L. Schneider and M. Becker. Advanced tuning methods and black box optimization. In B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors, *Applied Machine Learning Using mlr3 in R*. CRC Press, 2024. URL https://mlr3book.mlr-org.com/advanced_tuning_methods_and_black_box_optimization.html. [p366]
- H. Schulz-Kämpel, S. Fischer, R. Hornung, A.-L. Boulesteix, T. Nagler, and B. Bischl. Constructing Confidence Intervals for ‘the’ Generalization Error – a Comprehensive Benchmark Study, 2025. [p360]
- L. S. Shapley. A Value for n-Person Games. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games (AM-28), Volume II*, pages 307–318. Princeton University Press, Dec. 1953. ISBN 978-1-4008-8197-0. doi: 10.1515/9781400881970-018. [p358, 361]
- I. Sobol’. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and Computers in Simulation*, 55(1–3):271–280, 2001. ISSN 03784754. doi: 10.1016/S0378-4754(00)00270-6. [p358]
- C. Strobl, A.-L. Boulesteix, T. Kneib, T. Augustin, and A. Zeileis. Conditional variable importance for random forests. *BMC Bioinformatics*, 9(1):307, 2008. ISSN 1471-2105. doi: 10.1186/1471-2105-9-307. [p359, 360]
- J. Thomas. Preprocessing. In B. Bischl, R. Sonabend, L. Kotthoff, and M. Lang, editors, *Applied Machine Learning Using mlr3 in R*. CRC Press, 2024. URL <https://mlr3book.mlr-org.com/preprocessing.html>. [p364]
- C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 847–855. Association for Computing Machinery, 2013. ISBN 978-1-4503-2174-7. doi: 10.1145/2487575.2487629. [p360]
- M. J. van der Laan, E. C. Polley, and A. E. Hubbard. Super learner. *Statistical Applications in Genetics and Molecular Biology*, 6(1), 2007. doi: 10.2202/1544-6115.1309. [p360]
- D. S. Watson and M. N. Wright. Testing conditional independence in supervised learning algorithms. *Machine Learning*, 110(8):2107–2129, 2021. ISSN 1573-0565. doi: 10.1007/s10994-021-06030-6. [p358, 359, 361, 363, 367]

- D. S. Watson, K. Blesch, J. Kapar, and M. N. Wright. Adversarial Random Forests for Density Estimation and Generative Modeling. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, pages 5357–5375. PMLR, 2023. URL <https://proceedings.mlr.press/v206/watson23a.html>. [p359, 361, 366]
- B. Williamson and J. Feng. Efficient nonparametric statistical inference on population feature importance using Shapley values. In *Proceedings of the 37th International Conference on Machine Learning*, pages 10282–10291. PMLR, Nov. 2020. URL <https://proceedings.mlr.press/v119/williamson20a.html>. [p361, 363]
- B. D. Williamson, P. B. Gilbert, N. R. Simon, and M. Carone. A general framework for inference on algorithm-agnostic variable importance. *Journal of the American Statistical Association*, 118(543): 1645–1658, 2023. ISSN 0162-1459. doi: 10.1080/01621459.2021.2003200. [p359, 361, 363]

Lukas Burk

1) Leibniz Institute for Prevention Research and Epidemiology - BIPS 2) LMU Munich, Department of Statistics
3) Munich Center for Machine Learning (MCML) 4) University of Bremen
Achterstraße 30, 28359 Bremen, Germany
<https://lukasburk.de>
ORCID: 0000-0001-7528-3795
burk@leibniz-bips.de

Fiona Katharina Meinzer

1) LMU Munich, Department of Statistics 2) Munich Center for Machine Learning (MCML)
Department of Statistics
Chair of Statistical Learning and Data Science
ORCID: 0009-0002-6372-3401
fiona.ewald@lmu.de

Giuseppe Casalicchio

1) LMU Munich, Department of Statistics 2) Munich Center for Machine Learning (MCML)
Department of Statistics
Chair of Statistical Learning and Data Science
ORCID: 0000-0001-5324-5966

Marvin N. Wright

1) Leibniz Institute for Prevention Research and Epidemiology - BIPS 2) University of Bremen
Achterstraße 30, 28359 Bremen, Germany
ORCID: 0000-0002-8542-6291

Bernd Bischl

1) LMU Munich, Department of Statistics 2) Munich Center for Machine Learning (MCML)
Department of Statistics
Chair of Statistical Learning and Data Science
ORCID: 0000-0001-6002-6980